

Assignment 4 GenAI Analysis

Cesar Olono, EECS 348, 10/06/2026

a. GenAI programs used

Claude Sonnet
ChatGPT

b. How I accessed them

Opened claude.ai in my browser and typed the prompt into a new chat.
Opened [Chatgpt.com](https://chatgpt.com) and pasted the same prompt into a new chat.

c. Prompt used

Write a single-file C++ program named Assignment4.cpp that solves Sudoku puzzles.
Requirements: - Use an object-oriented design with classes (not standalone functions). - The solver must be recursive and use depth-first search with backtracking: find an empty cell, work out which digits 1-9 are valid there (not already in the same row, column, or 3x3 block), try each one, recurse, and undo the move if it fails. - If a puzzle has more than one solution, print all of them. - If a puzzle has no solution, print "No solution found". Input format: - The program reads five files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt from the current directory. - Each file contains [describe: e.g., 9 lines of 9 characters / digits separated by spaces]. A filled cell is a digit 1-9 and a blank cell is an underscore (_). - Here is an example file:

```
5 _ _ _ _ 1 7 _  
1 _ 6 5 _ 9 _ 4 _  
4 7 2 1 _ 6 _ _ _  
9 _ _ _ _ 5 _ _  
_ 1 8 _ 9 5 4 _ _  
6 _ _ 4 _ 2 3 8 9  
_ 4 _ _ _ 9 3 _  
_ 9 _ 7 _ 3 _ 5 _  
2 6 3 9 5 8 7 1 4
```

Output format: For each file, print: 1. The file name (e.g., puzzle1.txt) 2. The original puzzle 3. Each solution, or "No solution found" The program must compile with g++ on Linux.

d. Generated code

ChatGPT:

```
// Assignment4.cpp

#include <iostream>

#include <fstream>

#include <vector>

#include <string>


using namespace std;


class SudokuBoard {

private:

    int grid[9][9];


public:

    SudokuBoard() {

        for (int row = 0; row < 9; ++row) {

            for (int col = 0; col < 9; ++col) {

                grid[row][col] = 0;

            }

        }

    }


    bool loadFromFile(const string& filename) {
```

```
ifstream input(filename);
```

```
if (!input.is_open()) {
```

```
    return false;
```

```
}
```

```
string value;
```

```
for (int row = 0; row < 9; ++row) {
```

```
    for (int col = 0; col < 9; ++col) {
```

```
        if (!(input >> value)) {
```

```
            return false;
```

```
        }
```

```
        if (value == "_") {
```

```
            grid[row][col] = 0;
```

```
        }
```

```
        else if (value.length() == 1 &&
```

```
            value[0] >= '1' && value[0] <= '9') {
```

```
                grid[row][col] = value[0] - '0';
```

```
            }
```

```
        else {
```

```
            return false;
```

```
        }
```

```

    }
}

return true;
}

void print() const {
    for (int row = 0; row < 9; ++row) {
        for (int col = 0; col < 9; ++col) {
            if (grid[row][col] == 0) {
                cout << "_";
            }
            else {
                cout << grid[row][col];
            }

            if (col < 8) {
                cout << " ";
            }
        }
        cout << endl;
    }
}

```

```
bool isValid(int row, int col, int digit) const {
```

```
    // Check the row.
```

```
    for (int c = 0; c < 9; ++c) {
```

```
        if (grid[row][c] == digit) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    // Check the column.
```

```
    for (int r = 0; r < 9; ++r) {
```

```
        if (grid[r][col] == digit) {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    // Check the 3x3 block.
```

```
    int blockStartRow = (row / 3) * 3;
```

```
    int blockStartCol = (col / 3) * 3;
```

```
    for (int r = blockStartRow; r < blockStartRow + 3; ++r) {
```

```
        for (int c = blockStartCol; c < blockStartCol + 3; ++c) {
```

```
            if (grid[r][c] == digit) {
```

```
                return false;
```

```
            }
```

```

        }
    }

    return true;
}

bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == 0) {
                return true;
            }
        }
    }

    return false;
}

void setCell(int row, int col, int digit) {
    grid[row][col] = digit;
}

int getCell(int row, int col) const {
    return grid[row][col];
}

```

```
}  
};
```

```
class SudokuSolver {  
private:  
    SudokuBoard& board;  
    int solutionCount;  
  
    // Recursive depth-first search with backtracking.  
    void solveRecursive() {  
        int row;  
        int col;  
  
        // Find an empty cell.  
        if (!board.findEmptyCell(row, col)) {  
            // No empty cells means a complete solution was found.  
            ++solutionCount;  
  
            cout << "Solution " << solutionCount << ":" << endl;  
            board.print();  
            cout << endl;  
  
            return;  
        }  
    }  
};
```

```

    }

    // Try every possible digit from 1 through 9.
    for (int digit = 1; digit <= 9; ++digit) {
        if (board.isValid(row, col, digit)) {
            // Make the move.

            board.setCell(row, col, digit);

            // Recursively solve the resulting puzzle.

            solveRecursive();

            // Undo the move if the recursive search did not

            // permanently use this value.

            board.setCell(row, col, 0);
        }
    }
}

```

public:

```

    explicit SudokuSolver(SudokuBoard& sudokuBoard)
        : board(sudokuBoard), solutionCount(0) {
    }

```

```

    void solve() {

```

```
    solutionCount = 0;

    solveRecursive();

    if (solutionCount == 0) {
        cout << "No solution found" << endl;
    }
}

};
```

```
class Assignment {

private:
    vector<string> filenames;

public:
    Assignment() {
        filenames.push_back("puzzle1.txt");
        filenames.push_back("puzzle2.txt");
        filenames.push_back("puzzle3.txt");
        filenames.push_back("puzzle4.txt");
        filenames.push_back("puzzle5.txt");
    }

    void run() {
```

```

for (const string& filename : filenames) {

    cout << "=====" << endl;

    cout << filename << endl;

    cout << "=====" << endl;


    SudokuBoard board;


    if (!board.loadFromFile(filename)) {

        cout << "Error: Could not read a valid puzzle from "

            << filename << endl;

        cout << endl;

        continue;

    }


    cout << "Original puzzle:" << endl;

    board.print();

    cout << endl;


    cout << "Solutions:" << endl;


    SudokuSolver solver(board);

    solver.solve();


    cout << endl;

```

```
    }  
    }  
};
```

```
int main() {  
    Assignment assignment;  
    assignment.run();  
  
    return 0;  
}
```

Claude Response:

```
// Assignment4.cpp  
  
// Recursive Sudoku solver (depth-first search with backtracking).  
  
// Reads puzzle1.txt ... puzzle5.txt from the current directory and prints  
// every solution for each puzzle, or "No solution found".  
  
//  
  
// Compile: g++ -std=c++11 -Wall -Wextra -o Assignment4 Assignment4.cpp  
  
#include <fstream>  
  
#include <iostream>  
  
#include <string>  
  
#include <vector>
```

```

// -----

// SudokuBoard: owns the 9x9 grid and knows the Sudoku rules.

// A cell value of 0 means "empty" internally; '_' is used for I/O.

// -----

class SudokuBoard {

public:

    static const int SIZE = 9;

    static const int BLOCK = 3;

    SudokuBoard() : grid_(SIZE, std::vector<int>(SIZE, 0)) {}

    // Load a puzzle from a file. Returns false if the file can't be opened
    // or doesn't contain 81 valid cells.

    bool loadFromFile(const std::string& filename) {

        std::ifstream in(filename.c_str());

        if (!in) {

            return false;

        }

        int count = 0;

        char ch;

        while (count < SIZE * SIZE && in.get(ch)) {

            if (ch == '_') {

```

```

        grid_[count / SIZE][count % SIZE] = 0;

        ++count;

    } else if (ch >= '1' && ch <= '9') {

        grid_[count / SIZE][count % SIZE] = ch - '0';

        ++count;

    }

    // whitespace and anything else is skipped

}

return count == SIZE * SIZE;

}

int get(int row, int col) const { return grid_[row][col]; }

void set(int row, int col, int value) { grid_[row][col] = value; }

// Find the first empty cell scanning row by row. Returns false if none.

bool findEmpty(int& row, int& col) const {

    for (row = 0; row < SIZE; ++row) {

        for (col = 0; col < SIZE; ++col) {

            if (grid_[row][col] == 0) {

                return true;

            }

        }

    }

}

return false;

```

```
}
```

// Is 'digit' allowed at (row, col)? Checks row, column and 3x3 block.

```
bool isValidPlacement(int row, int col, int digit) const {  
    for (int i = 0; i < SIZE; ++i) {  
        if (grid_[row][i] == digit) return false; // same row  
        if (grid_[i][col] == digit) return false; // same column  
    }  
    int blockRow = (row / BLOCK) * BLOCK;  
    int blockCol = (col / BLOCK) * BLOCK;  
    for (int r = blockRow; r < blockRow + BLOCK; ++r) {  
        for (int c = blockCol; c < blockCol + BLOCK; ++c) {  
            if (grid_[r][c] == digit) return false; // same block  
        }  
    }  
    return true;  
}
```

// Do the pre-filled cells already break a rule? (e.g. two 5s in a row)

```
bool givensAreConsistent() {  
    for (int r = 0; r < SIZE; ++r) {  
        for (int c = 0; c < SIZE; ++c) {  
            int digit = grid_[r][c];  
            if (digit == 0) continue;
```

```

        grid_[r][c] = 0; // temporarily remove to test against the rest

        bool ok = isValidPlacement(r, c, digit);

        grid_[r][c] = digit;

        if (!ok) return false;
    }
}

return true;
}

```

```

void print(std::ostream& out) const {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (c > 0) out << ' ';

            if (grid_[r][c] == 0) out << '_';

            else out << grid_[r][c];
        }

        out << '\n';
    }
}

```

private:

```

    std::vector<std::vector<int> > grid_;

};

```

```

// -----
// SudokuSolver: recursive DFS + backtracking. Collects every solution.
// -----

class SudokuSolver {
public:
    // Solve a copy of the given puzzle; returns all solutions found.
    const std::vector<SudokuBoard>& solveAll(const SudokuBoard& puzzle) {
        solutions_.clear();

        SudokuBoard working = puzzle;

        if (working.givensAreConsistent()) {
            solve(working);
        }

        return solutions_;
    }

private:
    std::vector<SudokuBoard> solutions_;

    void solve(SudokuBoard& board) {
        int row, col;

        if (!board.findEmpty(row, col)) {
            solutions_.push_back(board); // no empty cells: a full solution

            return;
        }
    }

```

```

        for (int digit = 1; digit <= 9; ++digit) {
            if (board.isValidPlacement(row, col, digit)) {
                board.set(row, col, digit); // try it
                solve(board);               // recurse
                board.set(row, col, 0);     // undo (backtrack)
            }
        }

        // Keep going after a success so that *all* solutions are found.
    }
};

// -----

// PuzzleRunner: handles files and output formatting for each puzzle.

// -----

class PuzzleRunner {
public:
    void run(const std::string& filename) {
        std::cout << filename << "\n";

        SudokuBoard puzzle;

        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Error: could not read a valid puzzle from " << filename <<
"\n\n";

            return;

```

```

    }

    std::cout << "Original puzzle:\n";

    puzzle.print(std::cout);

    std::cout << "\n";

    SudokuSolver solver;

    const std::vector<SudokuBoard>& solutions = solver.solveAll(puzzle);

    if (solutions.empty()) {
        std::cout << "No solution found\n";
    } else {
        for (size_t i = 0; i < solutions.size(); ++i) {
            std::cout << "Solution " << (i + 1) << ":\n";

            solutions[i].print(std::cout);

            std::cout << "\n";
        }
    }

    std::cout << "\n";
}

};

int main() {

    PuzzleRunner runner;

```

```

    for (int i = 1; i <= 5; ++i) {

        runner.run("puzzle" + std::to_string(i) + ".txt");

    }

    return 0;

}

```

Use a monospace font (Courier New or Consolas) and a small size so the code stays readable. If a listing runs long, a page break before each one keeps things tidy.

e. Analysis

Program 1 is from ChatGPT. Program 2 is from Claude.

Correctness

Both programs use recursion with backtracking and keep going after a solution is found, so both print every solution. ChatGPT reads the file with input >> value, so it only works if the cells have spaces between them. Claude reads one character at a time in loadFromFile and skips everything except digits and _, so it works either way. Claude also has givesAreConsistent(), which catches puzzles that already break a rule. ChatGPT has no check like that. Claude's program matched solution1.txt and gave 1, 1, 1, none, and 2 solutions on the five puzzles.

Execution time

Claude's original code ran in 0.015 seconds.

Space complexity

Both programs change one board in place and undo each move with a set back to 0, so neither copies the board during the search. ChatGPT uses a plain 9x9 array and prints each solution right away, so it stores none. Claude uses a vector of vectors and saves every solution in the solutions_ vector. For a 9x9 board the difference is small, but ChatGPT uses a little less memory.

Maintainability

Both programs are split into three classes. Claude uses constants like SIZE and BLOCK instead of typing 9 and 3 everywhere, and it has a separate PuzzleRunner class for files and output. ChatGPT types the numbers directly. One comment in its solveRecursive method says a move is only undone sometimes, but the code always undoes it, which is confusing.

f. Selection and justification

I picked Claude. First, it loads puzzle files more reliably because it reads one character at a time. Second, `givensAreConsistent()` catches bad puzzles that ChatGPT would miss. Third, its named constants and separate classes make it easier to read and change. The extra memory it uses to store solutions is small.

g. Improvements

Recursive calls before and after:

puzzle1.txt: before 167, after 40
puzzle2.txt: before 31, after 29
puzzle3.txt: before 57, after 33
puzzle4.txt: before 17, after 8
puzzle5.txt: before 1269, after 61

I made 4 changes to claude's program

I made `loadFromFile` stricter. It now returns false if a file has more than 81 cells or any character other than the digits 1 to 9, `_`, and whitespace. Before, those were skipped without any warning. After the change, all five puzzles gave the same results: one solution for puzzles 1, 2 and 3, no solution for puzzle 4, and two solutions for puzzle 5. Puzzle 1 also matched `solution1.txt`.

Execution time

I added `countCandidates` and `findBestEmpty`, so the solver picks the empty cell with the least digits instead of the first empty cell. That means less wrong guesses, and the recursive calls dropped on every puzzle, most of all on puzzle 5 (1269 down to 61). The run time stayed about the same (0.015 seconds before and 0.017 after), because the puzzles are small and each call now does more work checking cells.

Space complexity

The solver used to save every solution in a `solutions_` vector. Now it prints each solution as soon as it finds it and only keeps a count, so it holds less memory, especially for puzzles with a lot of solutions.

Maintainability

I deleted two methods nothing used anymore (`findEmpty` and `get`), fixed comments that no longer matched the code, and I added a prolog and a comment on every block of code.